

LAB 2

System Calls in OS161

Programmazione di Sistema (IA-ZZ)

2024/25

Giorgio Cora

Obiettivi:

- Eseguire un programma utente
- Implementare le system calls: read, write ed exit

- In os161, i programmi utente possono essere lanciati usando il comando "p <percorso_del_programma>"
- Come esempio, provate ad eseguire il programma palin, che si trova nella cartella testbin (oppure cat nella cartella bin).
- Nel momento in cui viene lanciato un processo utente, l'eventuale chiamata ad una system call causa una trap nel kernel, la cui gestione è effettuata da una struttura specifica: il **trapframe**.

- ATTENZIONE! Non tutti i programmi utente possono essere eseguiti correttamente, in quanto manca il supporto per alcune funzionalità specifiche. In particolare:
 1. System Calls (interfacce tra user e kernel space)
 2. Gestione della memoria virtuale (allocazione e deallocazione)
 3. Argomenti (argc/argv)
- Oltre a quanto elencato, ciò che manca sono anche le direttive di sincronizzazione tra processi kernel ed utente. Questo può portare alla stampa di caratteri a schermo non voluti e/o non previsti.

```
Unknown syscall 55  
Unknown syscall 55  
Unknown syscall 55  
Unknown syscall 55  
Unknown syscall 55  
Unknown syscall 55  
Unknown syscall 55  
Unknown syscall 55  
Unknown syscall 55  
Unknown syscall 55  
Unknown syscall 55  
Unknown syscall 55  
Unknown syscall 55  
Unknown syscall 55  
Unknown syscall 55  
Unknown syscall 55  
Unknown syscall 3
```

Vengono effettuate le chiamate a SYS_write e SYS__exit ma non sono implementate

```
Fatal user mode trap 4 sig 10 (Address error on load e00f)  
panic: I don't know how to handle this  
sys161: trace: software-requested debugger stop  
sys161: Waiting for debugger connection...
```

L'elenco delle system calls ed il relativo codice identificativo si trova in:
/kern/include/kern/syscall.h

La gestione del trapframe si trova in:
/kern/arch/mips/syscall/syscall.c

- Le system calls devono essere implementate in kern/syscall.
- La read e la write possono essere implementate in maniera molto basilare, sfruttando le funzioni getch() e putch().
- Per il momento è sufficiente limitare la lettura allo STDIN e la scrittura a STDOUT ed STDERR (eventuali modifiche sono richieste nel caso in cui si voglia gestire la lettura e la scrittura da file).
- La funzione exit deve poter eseguire sia la rimozione dell'address space associato al processo utente, sia la distruzione del thread associato al processo utente stesso.

- Tutte le funzioni possono essere implementate in un singolo file (situato in `kern/syscall`).
- **IMPORTANTE!** Ogni volta che viene implementata una nuova system call è necessario andare ad aggiornare il file `/kern/arch/mips/syscall/syscall.c` aggiungendo nuove casistiche per le nuove system calls che sono state implementate.
- Ogni volta che una nuova system call viene aggiunta in `/kern/arch/mips/syscall/syscall.c` allora devo anche specificare quali sono i parametri che devono essere passati a questa funzione. Questi sono a decisione dell'utente.
- I nomi delle system calls li potete trovare in `/kern/include/kern/syscall.h`. Attenzione perchè `SYS__exit` ha un doppio underscore.